

Adding *Titanic: Adventure Out of Time* to ScummVM

A roadmap and scope assessment for the CyberFlix runtime

Reverse-engineering notes and implementation plan

June 9, 2026

Abstract

This document scopes the work required to run *Titanic: Adventure Out of Time* (CyberFlix, 1996/1997) under ScummVM. The findings below are based on direct inspection of the two retail Windows CD images. The title is built on CyberFlix’s proprietary, Mac-authored “*Bicycle*”-class runtime (internal signature LPPALPPA), the same engine family that powers *Dust: A Tale of the Wired West* and *RedJack: Revenge of the Brethren*. This is a **from-scratch engine**, not a variant of an existing ScummVM engine. It is unrelated to the existing `engines/titanic` (*Starship Titanic*), so a new engine plugin is required—the proposed name is `cyberflix`.

1 Summary and recommendation

Implementing this title is a large, multi-month reverse-engineering effort comparable in size to other “new proprietary engine” additions to ScummVM (e.g. Buried, Nancy, Hypno). It decomposes into seven substantial subsystems (archive container, bytecode VM, palettised graphics, sprite/shape codec, node-based room navigation, a custom movie codec, and audio), plus the standard ScummVM plumbing. None of these can be borrowed wholesale from existing engines.

Recommended sequencing: build bottom-up and keep something runnable at every step. Land the archive loader and a standalone asset dumper first (this de-risks every later phase), then the script VM, then graphics, then movies and audio, then glue it into a playable navigation loop. A vertical slice (boot → first room → navigate → play one movie) should be the first milestone that proves the architecture end-to-end.

2 What the game data looks like

2.1 Distribution and footprint

- Two CDs (`TITANIC1.iso` ≈635 MiB, `TITANIC2.iso` ≈620 MiB), ISO-9660, mastered on Mac (Toast).
- Both discs carry `TITANIC.EXE` (WinG-based, 32-bit, “(C) Copyright 1995 Cyberflix”), an `INSTALL` folder, and a `DATA` directory. The game refuses to run directly off CD; the user copies an `Install` folder to disk and streams the rest from whichever CD is inserted. ScummVM must therefore support **multi-disc/CD-swap** asset location.
- Per-disc asset inventory (disc 1): 112 `.MOV`, 30 `.TRK`, 23 `.SET`, 15 `.STG`, 13 `.SFX`, 11 `.SHP`, 2 `.CST`; disc 2 is dominated by `.SET` rooms and per-puzzle subfolders (`BOIL`, `BRIDGE`, `FUSE`, `TURBINE`, ...).

2.2 The LPPALPPA container format (verified)

Every asset file (and BOOTFILE) shares one container:

Offset	Field	Notes
0x00	u32 0x00010000	format/version magic
0x04	u32 file length	matches on-disk size exactly (LE)
0x10	u32 first-section size	e.g. 0x80/0x180
0x14	u32 resource count	e.g. 0xF7=247 for a .SET
0x20	"LPPALPPA"	signature ("APPL" word-reversed, twice)
0x200+	resource offset table	array of u32 chunk offsets
0x440+	header path + offset table	Mac HFS authoring path embedded

Critical finding—endianness. The payload is **big-endian**, authored on Mac. Decoding any header by reversing each 4-byte word yields a clean HFS path, e.g. BOOTFILE contains "Internal:new converts for Ian:bootfile.Temp" and CTL.STG contains "...:ctl.stg.Temp". The loader must byte-swap (or read big-endian) throughout. Embedded text strings are likewise stored in 4-byte-reversed groups.

Resource directory (verified on BEDSIT1.SET, 247 chunks). The offset table yields chunks each prefixed by a small record [u32 id] [u32 length] [u32 flags/count]. IDs run sequentially (0x10, 0x11, ...); early chunks are tiny metadata (8–0x74 bytes: palette, navigation, headers), then fixed 0x444-byte chunks (palette tables), then large variable chunks (21 KiB+) holding the compressed panorama/graphic tiles.

2.3 Asset type taxonomy (observed)

Ext	Role
BOOTFILE	Boot script + global variable symbol table (clock, phase, mission, playerdeath, savestage1..3, themevolume, navigation keys north/east/west, ...).
.STG	"Stage": a UI/screen state with an attached script (symbol table + compiled bytecode). CTL.STG = the control/HUD stage (open/save/quit/help/credits, subtoggle, navtoggle).
.SET	A location/room: node-based 360° navigation data, palettes, and many compressed image tiles (the largest assets, up to 33 MiB e.g. DECKBD2.SET).
.CST	"Cast": actor sprite collections (GANG.CST ≈20 MiB, characters/animation).
.SHP	"Shape": sprite/bitmap sets (inventory INVEN.SHP, UI HOUSE.SHP, minigame art).
.TRK	"Track": audio and/or animation timelines (music themes, room ambience, radio).
.MOV	Movies: named cels/frames ("BOMB 1 B", "FUZO", "briefcase") + a custom CyberFlix video codec (not Quick-Time/Smacker/Cinepak despite the .MOV extension).
.SFX	Sound effects.

2.4 The scripting VM (observed)

BOOTFILE and every .STG/.SET embed two cooperating sections: a **symbol table** (human-readable identifiers: state variables and event names such as `mousedown`, `keydown`, `knock`, `uparrow`, `seldir`, `advanceday`) followed by **compiled bytecode** in a narrow printable byte range (operators like `<` `%` `&` `[` `^` `)` `=`). This is a stack-based, event-driven interpreter: navigation, inventory, puzzle logic, and the global game state machine are all script-driven. Re-implementing this VM faithfully is the single highest-risk item in the project, because almost all game behaviour lives in data, not in native code.

3 Required ScummVM subsystems

- S1. Archive / resource layer.** Parse LPPALPPA, big-endian chunk directory, typed-chunk readers; a virtual file system over the installed folder + the two CDs with disc-swap handling.
- S2. Script VM.** Symbol-table loader, bytecode disassembler, and a stack interpreter with the full opcode set; the global state machine, event dispatch, and the engine’s built-in command vocabulary.
- S3. Graphics core.** 8-bit palettised surface, palette management (per-room 0x444 palette chunks), software blitter, dirty-rect compositor on ScummVM’s `OSystem` backend. Confirm native resolution (WinG; likely 640×480).
- S4. Shape/sprite codec.** Decode .SHP/.CST compressed cels (RLE-class scheme **[exact compression TBD]**) into surfaces with transparency and hotspots.
- S5. Room / node navigation.** Decode .SET panorama tiles and the node graph; implement directional movement (compass), hotspot hit-testing, and view rendering.
- S6. Movie player.** Reverse the custom .MOV codec (named-cel + interframe deltas), wire frame timing and audio sync. **[codec is fully bespoke—largest single unknown]**.
- S7. Audio.** Decode .TRK/.SFX streams **[PCM/ADPCM TBD]**, mix via ScummVM’s `Audio::Mixer`, music themes with volume control (`themevolume`).
- S8. Engine glue & UX.** `MetaEngine` + `AdvancedDetector` tables (MD5s for both discs, language variants), save/load mapped onto the game’s `savestage` variables, the main loop, debugger console, `detection.tables.h`, `module.mk`, `configure.engine`, registration in `engines/engines.mk`, and POTFILES.

4 Phased plan

Ph.	Goal / exit criterion	Work
0	Scaffold builds. Empty <code>cyberflix</code> engine compiles, detects the game, opens a black window.	<code>MetaEngine</code> , detection table from real MD5s, <code>module.mk</code> , <code>configure.engine</code> , <code>engines.mk</code> , POTFILES, boilerplate <code>Engine</code> subclass + console.

1	Asset transparency. Standalone dumper lists every chunk in any container; round-trips BOOTFILE.	LPPALPPA parser, big-endian readers, chunk directory, symbol-table extractor; a <code>devtools/</code> dumper. Document the chunk-ID map per asset type.
2	Script VM. Boot script runs; global state machine ticks; events dispatch.	Bytecode disassembler → interpreter; opcode set; variable store; built-in commands; BOOTFILE + CTL.STG execute.
3	Graphics + one static room. First .SET renders with correct palette.	Palettised surface, palette chunks, .SHP decoder, SET tile decode, compositor.
4	Navigation slice. Boot → first room → turn N/E/S/W → enter adjacent room. (<i>Milestone: vertical slice.</i>)	Node graph, hotspots, cursor, compass UI driven by the VM.
5	Movies. One .MOV plays in-engine with audio.	Reverse the custom codec; frame pipeline; A/V sync.
6	Audio + inventory + HUD. Music themes, SFX, inventory (INVEN.SHP), control bar.	.TRK/.SFX decode + mixer; inventory model; CTL.STG HUD interactions.
7	Save/load + completion. Save anywhere; full game traversable; minigames (BOMB, BLK-JACK, ENIGMA, PUNCH-BAG...).	Map saves onto <code>savestage</code> vars; per-minigame scripts/assets; polish.
8	Compatibility & submission. GMM, keymaps, achievements/ports quirks, detection for other releases/languages, code review against ScummVM standards.	Testing matrix, translations, upstream PR.

5 Open questions / unknowns (to retire early)

- Native resolution and color depth (WinG implies 8-bit; confirm 640×480 vs 320×240).
- Exact .SHP/.CST and .SET-tile compression scheme.
- The .MOV codec (interframe model, key/delta frames, audio interleave). *Highest risk.*
- .TRK/.SFX audio encoding (raw PCM vs ADPCM, sample rate).
- Complete VM opcode set and built-in command semantics.
- Save-game schema (the `savestage1..3` variables and what they capture).
- Release variants for detection: Windows vs Mac, localized editions, later re-releases.

6 ScummVM integration checklist

- New plugin `engines/cyberflix/` (name TBD; avoids clash with `engines/titanic` = Starship Titanic).
- `detection_tables.h` with MD5/size/language entries for both discs.

- `configure.engine`, `engines.mk` (`ENABLE_CYBERFLIX`), `POTFILES`, `credits.pl`.
- **MetaEngine**: detection, save metadata, supported features, keymaps.
- Multi-disc handling consistent with ScummVM conventions (CD-swap prompts).
- Coding standards, `Common::Stream` usage, `debug()`/console, no external deps.

7 Risks

- **Bespoke movie codec** (S6): no reference; pure reverse engineering. Mitigation: tackle a tiny `.MOV` (e.g. `BOMB`) first; fall back to an external decode tool for validation.
- **VM completeness** (S2): missing opcodes silently break far-off puzzles. Mitigation: corpus-disassemble *all* scripts up front to enumerate the opcode/command set before interpreting.
- **Asset volume / CD-swap** (S1): ~ 1.3 GB across two discs; correctness of the virtual FS gates everything.
- **Scope**: full completion is multi-month. The phase 4 vertical slice is the realistic first deliverable and should be the gate for committing to the rest.